



Cascade

автономная команда разработки ПО

БТ → УС → НФТ → ФТ → код → тесты → README

100%

БТ → ФТ

12

файлов

\$0.02

/прогон

3.4 мин

end-to-end

github.com/RC6HEX/cascade

rc6hex.github.io/cascade · [live demo](#)



Разработка ПО — это не только код

почему один промпт ≠ полноценный проект

01

LLM пишут код

но не делают требования, архитектуру, тесты

02

Трассировка теряется

почему именно эта функция? откуда взялась?

03

Один промпт

≠ полноценный проект с документацией



Приложение, которое создаёт другие приложения

вход → каскад артефактов → готовый проект

ВХОД

Markdown-документы

БТ

бизнес-требования

БП

бизнес-процесс

Features

характеристики (опционально)

ВЫХОД

Готовый проект

src/

исходный код приложения

docs/

use-cases, НФТ, ФТ

tests/

Vitest unit-тесты

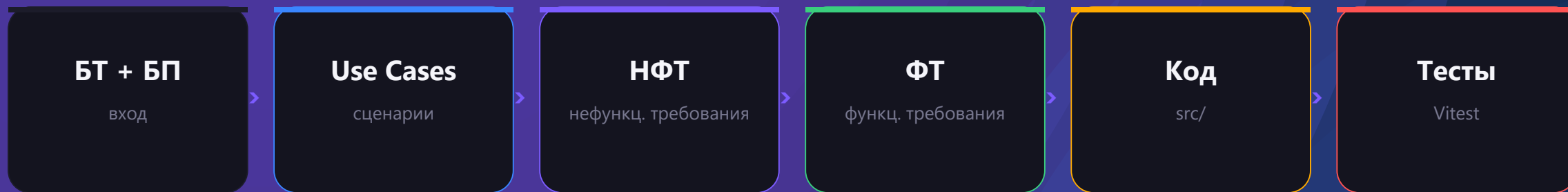
README.md

инструкция к запуску



Каскад артефактов

каждый уровень ссылается на предыдущий



Пример трассировки одного требования:





Архитектура

три слоя, чёткие границы

Web UI

FastAPI · SSE · vanilla JS

- POST /api/jobs
- GET .../stream (SSE)
- GET .../app/{path}
- GET .../traceability

pipeline.py

6 шагов с self-check

- use_cases · nfr · fr
- code (streaming)
- tests · readme
- self-check loops

llm.py

OpenRouter / Gemini

- retry с backoff (429, 5xx)
- streaming через httpx
- thread-safe Usage
- 12 моделей



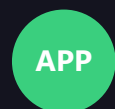
Стек

vanilla > магия



Python 3.11+

httpx + FastAPI + Uvicorn



Сгенерированные приложения

vanilla HTML + JS + CSS, без сборки



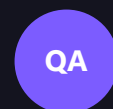
Web интерфейс

FastAPI + SSE + vanilla JS, без React/Vue



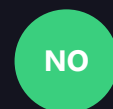
LLM провайдеры

OpenRouter (12 моделей) или Gemini API



Тесты сгенерированных

Vitest + happy-dom



Без LangChain / CrewAI

контроль над промптами > магия агентов

12 моделей через OpenRouter

переключаются live в Settings — без рестарта

Модель	\$ in / out (1M)	контекст	применение
● DeepSeek V3	\$0.27 / 1.10	64K	дефолт smart
● Qwen 2.5 72B	\$0.13 / 0.40	32K	дефолт fast
● DeepSeek R1	\$0.55 / 2.19	64K	reasoning
● Llama 3.3 70B	\$0.13 / 0.40	128K	универсальная
● MiniMax 01	\$0.20 / 1.10	1M	длинный контекст
● Gemini 2.5 Pro	\$1.25 / 10.00	1M	максимум качества
● Gemini 2.5 Flash	\$0.30 / 2.50	1M	баланс
● Claude 3.5 Haiku	\$0.80 / 4.00	200K	стабильно
● Qwen 2.5 Coder 32B	\$0.07 / 0.16	32K	код
● Mistral Small 3.2	\$0.10 / 0.30	96K	быстрая
● Qwen 2.5 7B	\$0.04 / 0.10	32K	стресс-тест
● Llama 3.2 3B	\$0.02 / 0.04	128K	отладка



Каждому шагу — свой эксперт

persona-driven prompts: модель пишет ФТ как продакт, код как сеньор

- | | | | |
|---|-----------|---------------------------------|-------------------------------------|
| 1 | use_cases | Senior Business Analyst, 15 лет | -> форматные UC с источниками БТ |
| 2 | nfr | Solution Architect | -> измеримые критерии с числами |
| 3 | fr | техлид + системный аналитик | -> обязательная трассировка БТ → ФТ |
| 4 | code | Senior Frontend Engineer L7 | -> @implements ФТ + кодекс качества |
| 5 | tests | QA-инженер с диким взглядом | -> ловит регрессии до релиза |
| 6 | readme | технический писатель | -> копиястные команды, без воды |



Self-check loops

главное усиление по ТЗ — резко поднимает качество

100%

обязательных БТ
покрыто ФТ

после ретраев

(в среднем — 1 ретрай на FR-step)

Как работает

- 1 Шаг сгенерирован
- 2 Validator парсит ID:
БТ → ФТ → @implements
- 3 Не покрыто? — feedback модели
- 4 Шаг переделывается, до 2 ретраев

ПРИМЕР ИЗ РЕАЛЬНОГО ПРОГОНА

FR-step: «missing БТ-05 в источниках»

→ retry 1 → ФТ-04 теперь ссылается на БТ-05



Streaming partial output

код пишется в UI на лету — не ждёшь 2 минуты

// live streaming

src/calculator.js · 1.2 KB

```
/**
 * @implements ФТ-01
 * @implements ФТ-04
 */
export class Calculator {
  constructor() {
    this.currentInput = '0';
    this.prevInput = null;
    this.history = [];
  }
  handleNumberInput(n) {
    if (this.resetNextInput) {
```



Параллелизация шагов

use_cases + nfr — независимые → запускаются одновременно

ДО

последовательно

use_cases

20s

nfr

20s

$\Sigma \approx 40$ секунд

ПОСЛЕ

ThreadPoolExecutor(2)

use_cases · 20s

nfr · 20s

$\Sigma \approx 20$ секунд

–50% времени на этих шагах



Refinement mode

короткий комментарий → точечные правки (по ТЗ)

ПРИМЕР

«поменяй цветовую схему на синюю»



изменён только `src/styles.css`

40s

длительность

1

LLM call

1

файл изменён

4

файла нетронуты



Traceability matrix

БТ -> УС -> ФТ -> @implements в одной картинке

5

БТ всего

4

обязательных

4

юз-кейсов

7

ФТ

100%

ФТ → @implements

матрица в UI · реальные данные из последнего прогона:

БТ	обязательность	УС	ФТ
БТ-01	обяз.	УС-01	ФТ-01
БТ-02	опц.	УС-02	ФТ-05, ФТ-06
БТ-03	обяз.	УС-02	ФТ-03, ФТ-04
БТ-04	обяз.	УС-01	ФТ-02
БТ-05	обяз.	УС-03	ФТ-07



Per-step model picker

модель на каждый из 6 шагов отдельно

use_cases

Llama 3.2 3B (debug)



fr

DeepSeek V3



tests

Qwen 2.5 7B



nfr

Qwen 2.5 72B



code

Claude 3.5 Haiku



readme

Mistral Small





Job history + cost estimator

ИСТОРИЯ ЗАПУСКОВ

Job persistence

- `_job_meta.json` на диск после каждого пайплайна
- `_scan_disk_jobs()` при startup восстанавливает всё
- `GET /api/jobs` — sorted by mtime
- Sticky panel в UI: status / task / files
- Replay сохранённых SSE-событий

СТОИМОСТЬ

Live cost estimator

пересчёт при каждом input в БТ/БП

\$0.05

≈ один полный прогон

- DeepSeek V3 + Qwen 72B — дефолт
- Видно сразу, какая модель сколько съест
- На дешёвых: \$0.02, на премиум: \$0.30
- Учитывает per-step переопределения



Качество кода — 23 unit-теста

pytest tests/ — 23/23 PASSED

23 / 23

PASSED

pytest tests/ · 0.10 сек весь suite

ЧТО ПОКРЫТО

test_parser.py

10 кейсов

разные header levels, backslashes,
дедупликация, пустые блоки, inner backticks

test_validators.py

13 кейсов

извлечение ID с разными типе,
парсинг русско/анг таблиц,
сортировка ID численно (ФТ-02 < ФТ-10)

Демо А — Веб-калькулятор

QuickCalc · реальный прогон через UI · 26 апр 2026

5

БТ

(4 обяз.)

4

юз-кейса

7

ФТ

100% покрыто

12

файлов

code + docs + tests

\$0.02

СТОИМОСТЬ

27К токенов

3.4 мин

время

с self-check

СГЕНЕРИРОВАННЫЕ ФАЙЛЫ

- `src/index.html` 1.7 KB · точка входа
- `src/app.js` 1.2 KB · bootstrap
- `src/calculator.js` 5.8 KB · 14 @implements
- `src/keyboard.js` 1.4 KB · 1 @implements
- `src/styles.css` 3.9 KB · тёмная тема
- `tests/calculator.test.js` 7.3 KB · 18 it()
- `tests/keyboard.test.js` 2.1 KB · 4 it()

ХРОНОМЕТРАЖ

- `use_cases` 31.9s OK · first try
- `nfr` 22.4s OK · first try
- `fr` 100.9s retry 1 — нашёл missing БТ-05
- `code` ~60s OK · 14 @implements
- `tests` ~50s OK · Vitest + happy-dom
- `readme` ~15s OK



Задания В + С

один пайплайн — три абсолютно разных приложения

В · СРЕДНЕЕ

TaskBoard

- Канбан-доска с тремя статусами
- Drag-and-drop между колонками
- Создание / удаление задач
- Фильтрация по статусу
- Сохранение в localStorage
- Подтверждение удаления

С · СЛОЖНОЕ

CurrencyX

- 12 валют (USD, EUR, RUB, GBP, JPY...)
- Курсы из открытого API без ключа
- Кеш в localStorage с TTL 1 час
- Мгновенный пересчёт по input
- Кнопка swap — поменять валюты местами
- Fallback: «курс от [дата]» при offline



Cascade

автономный генератор приложений

Итоги

что мы построили — в цифрах

4

волны разработки

23

API endpoint

12

LLM моделей

23 / 23

unit-теста pass

100%

БТ → ФТ покрытие

ЭТАПЫ

Wave 1

MVP pipeline + persona prompts + self-check + tests

Wave 2

refinement + parallelism + streaming + per-step models

Wave 3

live preview + traceability matrix + history + cost estimator

open-source • pull requests welcome

github.com/RC6HEX/cascade

rc6hex.github.io/cascade — live demo